Evolutionary Computation

Constraint Handling in Evolutionary Algorithms (II)

Shan He

School of Computer Science
University of Birmingham

Outline of Topics

1. Stochastic Ranking
2. Feasibility Rules
3. Repair Approach

Penalty Functions Demystified

- Penalty function essentially:
  - Transforms fitness
  - Changes rank $\rightarrow$ changes selection
- Why not change the rank directly in an EA?
- Stochastic Ranking

Stochastic Ranking

- Proposed by Dr Runarsson and Prof. Xin Yao in our school in 2000
- A **special rank-based selection scheme** that handles constraints
- There is no need to use any penalty functions
- It's self-adaptive since there are few parameters to set
- Became the one of the popular constraint handling techniques due to its effectiveness and simplicity, see the citations of the paper

Stochastic Ranking — Self-adaptive

Stochastic Ranking Algorithm

DO the following until there is no more change in the ranking.
FOR $i := 1$ TO $M - 1$ DO
    $u := U(0; 1)$, // u is a uniformly distributed random number;
    IF $G(x_i) = G(x_{i+1}) = 0$ OR $u \leq P_f$ THEN
        IF $f(x_i) > f(x_{i+1})$ THEN swap(x_i; x_{i+1});
    ELSE
        IF $G(x_i) > G(x_{i+1})$ THEN swap(x_i; x_{i+1});

M is the number of individuals, $G(x_{i+1})$ is the sum of constraint violation and P_f is a constant that indicates the probability of using the objective function for comparison in ranking.
Note: Essentially follow the bubble sort algorithm to rank the individuals.

The role of P_f

- $P_f > 0.5$:
  - Most comparisons are based on $f(x)$ only
  - Infeasible solutions are likely to occur
- $P_f < 0.5$:
  - Most comparisons are based on $G(x)$ only
  - Infeasible solutions are less like to occur, but the solutions might be poor
- In practice, P_f is often set between 0.45 and 0.5

Question: why we allow infeasible solutions to be selected?

Penalty Functions Demystified

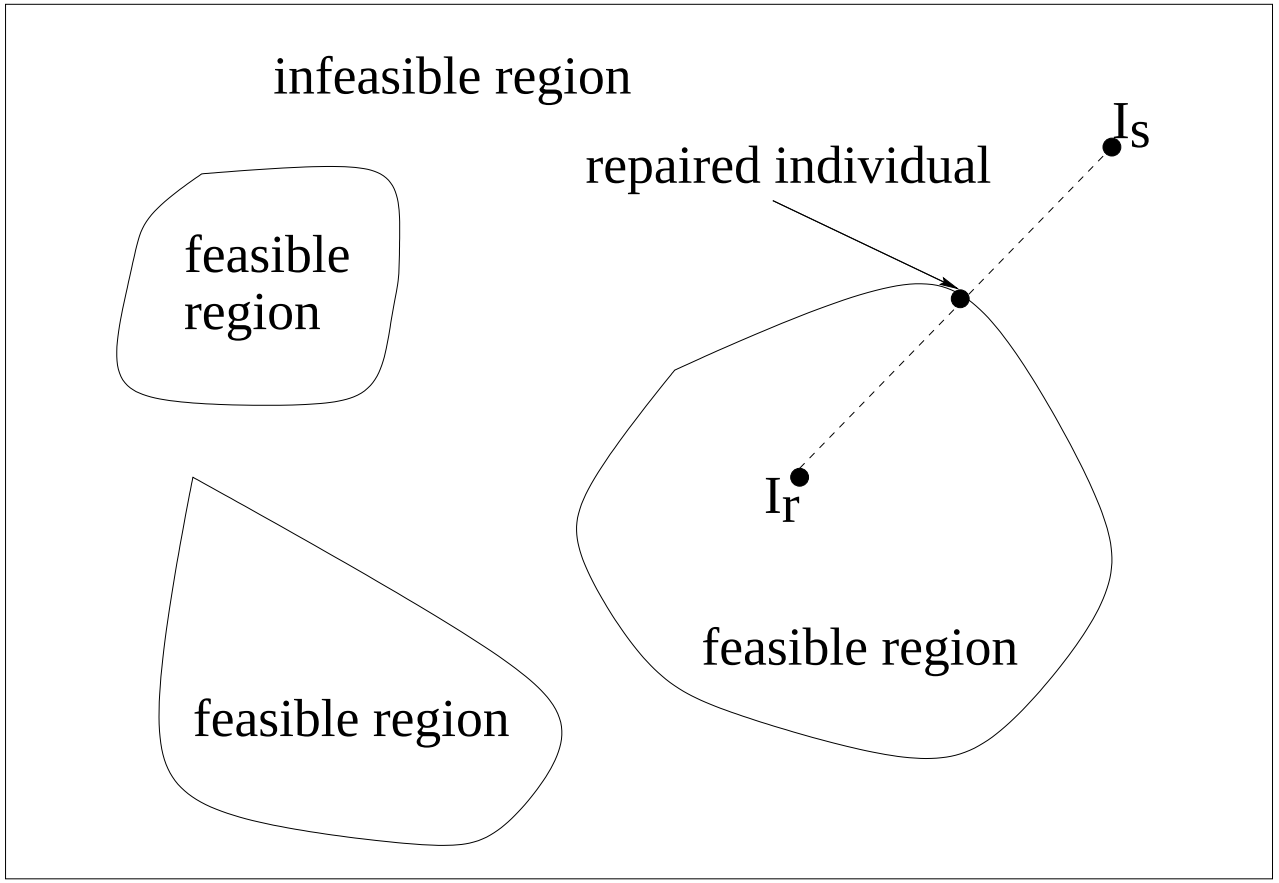
- Penalty function essentially performs:
  - Fitness transformation
  - Rank change $\longrightarrow$ selection change
- Why not change **selection** directly in an EA?

Feasibility rule

- Proposed by Deb in 2000
- Based on (binary) tournament selection
- After randomly choose k ($k = 2$) individuals to form a tournament, apply the following rules:
  - Between 2 feasible solutions, the one with better fitness value wins
  - Between a feasible and an infeasible solutions, the feasible one wins
  - Between 2 infeasible solutions, the one with lowest G wins
- Pros: simple and parameter free
- Cons: premature convergence (**Question**: Why? Can you improve the feasible rule method?)

Repair Approach to Constraint Handling

Instead of modifying an EA or fitness function, infeasible individuals can be "repaired" into feasible ones.

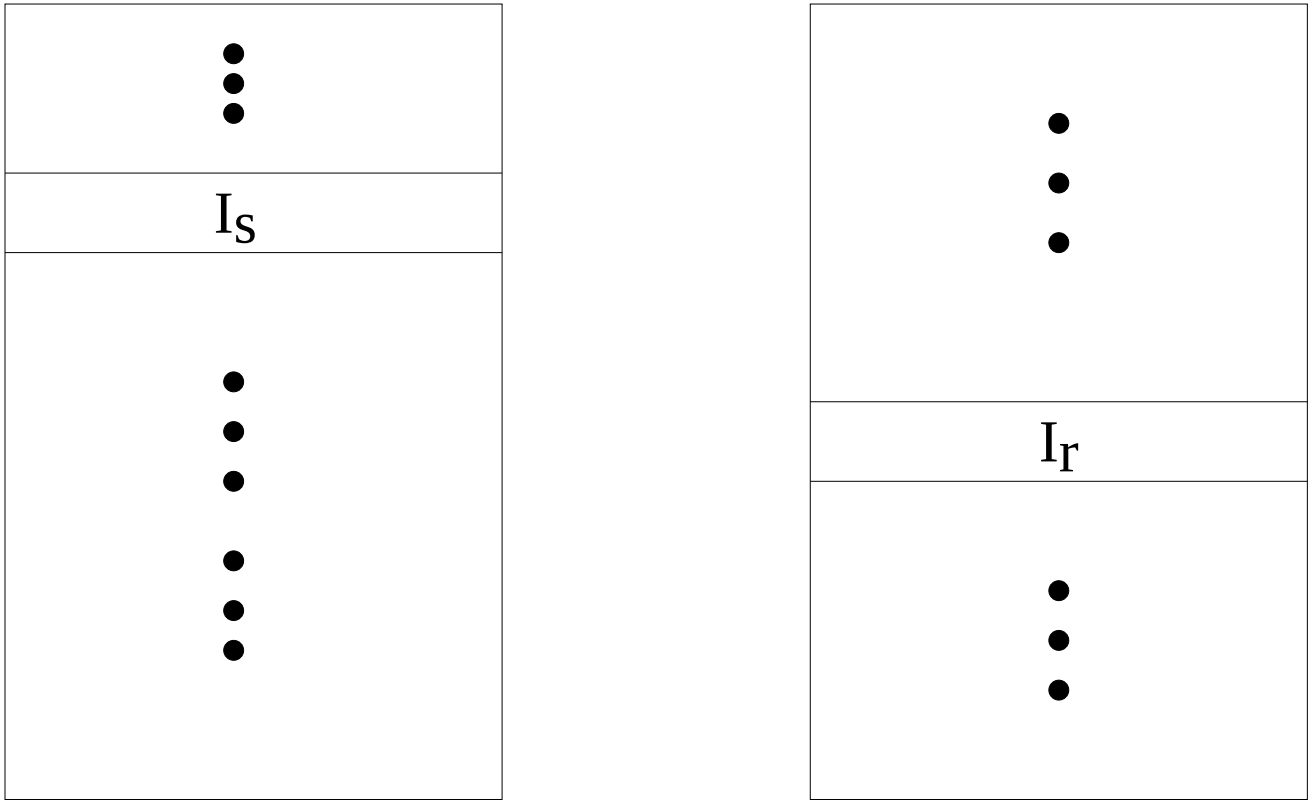


Repairing Infeasible Individuals

- Let I_s be an infeasible individual and I_r a feasible individual
- We maintain two populations of individuals:

population of evolving individuals (feasible or infeasible)

population of feasible reference individuals (changing but not evolving)



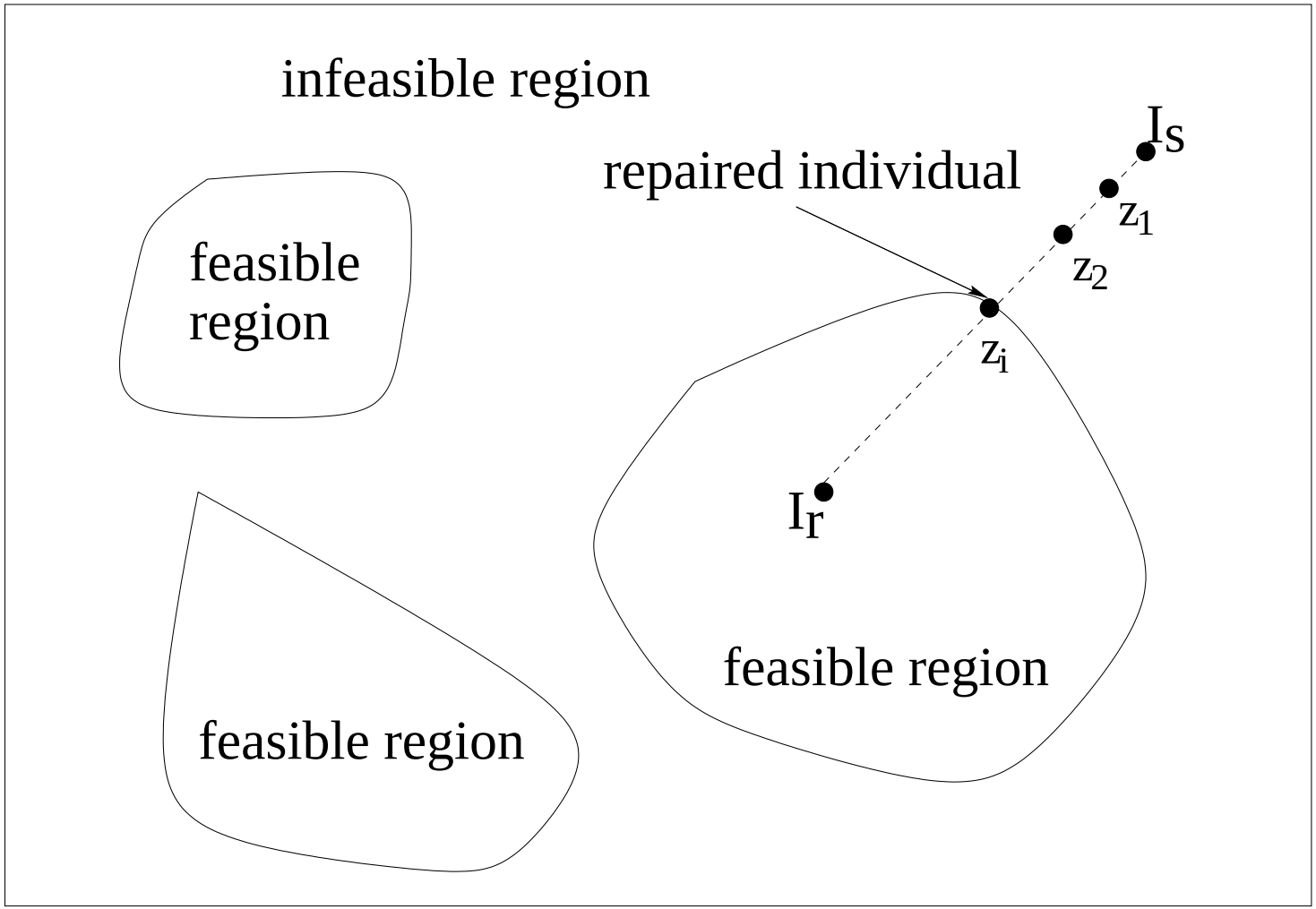
Repairing Algorithm

Repairing Algorithm

Select a reference individual I_r.
DO the following until individual z_i is feasible
$z_i = a_i I_s + (1 - a_i) I_r$ where $0 < a_i < 1$.
Calculate the fitness value of z_i: $f(z_i)$
IF $f(z_i) \leq f(I_r)$ THEN Replace I_s by z_i
ELSE
$u := U(0; 1)$, // u is a uniformly distributed random number;
IF $u \leq Pr$ THEN Replace I_s by z_i

Repairing Infeasible Individuals

Let I_s be an infeasible individual and I_r a feasible individual.



Repairing Algorithm: Implementation Issues

- How to find initial reference individuals?
  - Preliminary exploration
  - Human knowledge
- How to select I_r
  - Uniformly at random
  - According to the fitness of I_r
  - According to the distance between I_r and I_s
- How to determine a_i
  - Uniformly at random between 0 and 1
  - Using a fixed sequence, e.g., $\frac{1}{2}, \frac{1}{4}, \cdots$
- How to choose Pr: A small number, usually < 0.5

Conclusion

- Adding a penalty term to the objective function is equivalent to changing the fitness function, which is in turn equivalent to changing selection probabilities.
- It is easier and more effective to change the selection probabilities directly and explicitly: stochastic ranking and feasibility rules
- There are other constraint handling techniques such as repairing methods (see a review paper in the reading list)
- We have covered numerical constraints only. We have not dealt with constraints in a combinatorial space.

Further reading

- T. P. Runarsson, X. Yao, Stochastic ranking for constrained evolutionary optimization, IEEE Transactions on Evolutionary Computation, 4(3):284-294, September 2000.
- T. Runarsson and X. Yao, Search Bias in Constrained Evolutionary Optimization, IEEE Transactions on Systems, Man, and Cybernetics, Part C, 35(2):233-243, May 2005.
- E. Mezura-Montesa, C. A. Coello Coello, Constraint-handling in nature-inspired numerical optimization: Past, present and future. Swarm and Evolutionary Computation, 2011
- Interior-point method: essentially using a penalty method called Barrier function